

High-Level Design (HLD)

AI-Powered Visual Book Generator

EY GenAI Engineering Assignment — MVP

Aman Agrawal

~11+ years in software engineering · M.Tech in AI & ML, IIT Jodhpur

aman.agrawal@telus.com

linkedin.com/in/aman-agrawal007 · github.com/Aman007Agg/ai-powered-visual-book-generator

July 2026

Contents

1. Executive Summary
2. Problem Statement
3. Objectives & Scope
4. Functional Requirements
5. System Architecture (HLD)
6. Technology Stack & Rationale
7. AI Workflow & Prompt Strategy
8. API Design
9. Folder Structure
10. Assumptions & Trade-offs
11. Future Enhancements
12. Conclusion

1. Executive Summary

The AI-Powered Visual Book Generator is a browser-based application that converts raw book content — page text and images — into a visually designed first draft of a book. Rather than relying on fixed templates, it uses a Large Language Model (LLM) to reason about each page and recommend intelligent layouts.

For every page, the system generates two distinct layout options. The user selects the preferred one and continues page by page until a complete book draft is assembled. The central engineering principle is a clean separation of concerns: the LLM decides layout semantics as structured JSON, and the Angular front end owns all presentation. The model never emits HTML or CSS.

This delivers a secure, deterministic and testable system that produces context-aware layouts far faster than manual typesetting, while remaining simple and within the MVP scope defined by the PRD.

2. Problem Statement

Marketing and content teams routinely produce content-heavy documents that can run to hundreds of pages. Turning that raw content into well-designed, readable layouts is currently a manual effort:

- Designers spend considerable time arranging text and images.
- Producing a polished document takes multiple iterations.
- The overall turnaround is slow and expensive.

The opportunity is to automate the first version of layouting (typesetting) using an LLM — reducing manual effort while keeping the layout recommendations context-aware, so a non-designer can produce a high-quality first draft quickly.

3. Objectives & Scope

Objectives

- Browser-based application requiring no installation.
- Accept optional book-level context (title, objective).
- Accept page-level text content.
- Accept one or more images per page.
- Use AI to generate two intelligent layout options per page.
- Allow the user to select the preferred layout.
- Continue page by page until a complete book draft is produced.

In scope (MVP)

Page-by-page text/image input, AI-recommended layouts, two options per page, user selection, and an on-screen complete book draft.

Out of scope (PRD non-goals)

Advanced editing (drag-and-drop, voice), persistent storage, strict formatting/theming controls, and collaboration features.

4. Functional Requirements

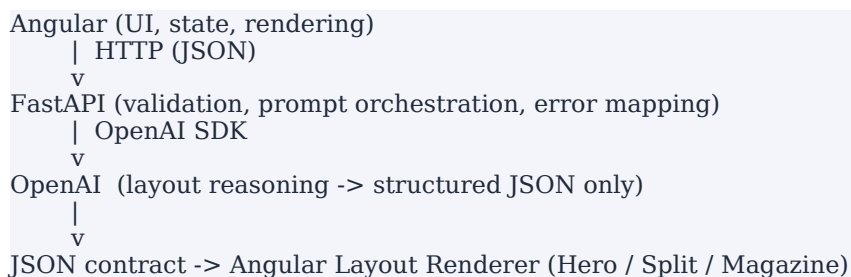
- FR1 — Create a new book with optional title and objective.
- FR2 — Enter page text (required) and upload images (optional).
- FR3 — Submit page content to the backend for AI layout generation.
- FR4 — Receive exactly two layout options, each with a type, description, confidence and ordered sections.
- FR5 — Render both options visually and let the user select one.
- FR6 — Commit the selected layout as a finished page.
- FR7 — Repeat for additional pages; finish to view the complete draft.
- FR8 — Navigate the finished book page by page, each rendered with its chosen layout.

Non-functional

- Security: the model never emits markup; all AI text is rendered via escaped interpolation.
- Reliability: structured JSON output is validated server-side; typed error mapping for OpenAI failures.
- Performance: page-at-a-time processing keeps each request within LLM context limits (~500-page books).

5. System Architecture (HLD)

The system has three layers with strict responsibilities. Angular owns UI, state and rendering; FastAPI owns validation, prompt orchestration and error mapping; OpenAI performs layout reasoning and returns structured JSON only.



Why the LLM never generates HTML/CSS:

- Security — no model-generated markup means no HTML/CSS injection (XSS) surface.
- Determinism & testability — rendering is a pure function of validated JSON.
- Cost — restyling (brand, spacing) never requires another model call.
- Portability — the same JSON could later drive PDF or print output.

Rendering strategy

A LayoutRenderer dispatcher selects a presentational component at runtime based on the AI's layout_type (hero / split / magazine), with a safe fallback to Hero. Each layout is a dumb, signal-input component; a shared pure function groups sections by role and a pure pipe resolves image references.

6. Technology Stack & Rationale

Layer	Technology	Rationale
Frontend	Angular 21 (standalone, Signals, zoneless), Angular Material, TypeScript 5.9	Modern component-driven SPA; Signals give simple, leak-free reactive state; strong typing.
Backend	FastAPI, Python, Pydantic v2, Uvicorn	High-performance async API with first-class validation — ideal for a typed AI contract.
AI	OpenAI Chat Completions, JSON-object output	Strong reasoning and structured JSON we can validate and render deterministically; vision-capable (roadmap).
Rendering	HTML, CSS Grid & Flexbox	Responsive, framework-native layout without any model-generated markup.
Testing	pytest (backend), Vitest (frontend)	Fast, deterministic tests around the schema, prompt, endpoint and state logic.

7. AI Workflow & Prompt Strategy

The request flows through six stages:

User input -> Backend validation -> Prompt construction ->
GPT layout reasoning -> Structured JSON -> Angular rendering

Prompt design

Prompts are treated as production software. The system prompt defines the role (editorial book-layout designer), the goal (exactly two recommendations), boundaries (no HTML/CSS — decide layout, section order and image placement only), and a strict output schema.

Constraints enforced

- layout_type must be one of: hero, split, magazine.
- The two options must use two DIFFERENT layout types (no cosmetic duplicates).
- confidence is an integer 0–100; image_reference is 0-based and bounded by the image count.

Safety net

The call uses response_format = json_object, and the response is validated by a Pydantic model. Malformed or out-of-contract output fails validation and is mapped to a clean error rather than reaching the UI.

Vision (scoped)

For the MVP the model receives the image count, not the pixels; image-content vision is documented as a scoped assumption with a one-line upgrade path to GPT-4o image inputs.

8. API Design

Health: GET / — returns service status.

Generate: POST /api/layout/generate — returns two layout options for one page.

Request (GenerateLayoutRequest)

```
{
  "title": "AI Fundamentals",      // optional
  "objective": "Teach beginners...", // optional
  "page_text": "...",             // required (min length 1)
  "images": [ { "fileName", "mimeType", "base64" } ]
}
```

Response (GenerateLayoutResponse)

```
{
  "page_summary": "...",
  "layout_options": [
    { "id", "layout_type", "layout_name", "description",
      "confidence", "sections": [ { type, text?, image_reference? } ] },
    { ... second option, different layout_type ... }
  ]
}
```

Error contract

HTTP	error_code	Cause
401	INVALID_API_KEY	OpenAI rejected the key
402	INSUFFICIENT_CREDITS	Quota/credits exhausted
429	RATE_LIMIT	Rate limited by OpenAI
400	CONTEXT_LENGTH	Input exceeds model context
422	(validation)	Request failed schema validation (e.g. empty text)
500	OPENAI_API_ERROR / MISSING_API_KEY	Other OpenAI / parse / config failure

9. Folder Structure

```
ai-powered-visual-book-generator
|-- backend
|   |-- app
|   |   |-- api/      health.py, layout.py
|   |   |-- core/     exceptions.py
|   |   |-- prompts/  layout_prompt.py
|   |   |-- schemas/  request_schema.py, response_schema.py
|   |   |-- services/ openai_service.py
|   |   |-- main.py   app, CORS, routers
|   |-- tests/        pytest suite
|   |-- requirements.txt
|-- frontend
|   |-- src/app
|   |   |-- core/services/ http (ApiService) + state (BookStateService)
|   |   |-- features/     home, new-book, page-input, generating,
|   |   |                   layout-preview, book-summary
|   |   |-- shared/
|   |   |   |-- components/renderers/ LayoutRenderer + Hero/Split/Magazine
|   |   |   |-- models/               typed contracts
|   |   |   |-- pipes/                ImageRefPipe
|   |   |   |-- utils/                groupSections()
|-- docs    ARCHITECTURE, API, PROMPTS, ASSUMPTIONS, DEMO, HLD
|-- README.md
```

10. Assumptions & Trade-offs

- Image vision is scoped: the model receives the image count, not pixels (documented, with a clear upgrade path).
- State is in-memory: no persistence, per the PRD non-goal; a refresh starts over.
- Text is required per page; images are optional enrichment.
- Page-at-a-time model calls: bounds context, at the cost of no cross-page style consistency (a future item).
- Deferred hardening: request timeouts, max_tokens, image size caps and structured logging were identified and deferred per 'functionality first'.
- Signals over a heavier store (e.g. NgRx): right-sized state for an MVP; zoneless-friendly; no boilerplate.

11. Future Enhancements

- Enable true image vision (GPT-4o image inputs).
- Edit layouts via natural language.
- Persistent storage — save and revisit books/pages.
- Multi-user collaboration.
- PDF export of the finished book.
- Global style consistency across pages; templates / themes.
- Batch processing / bulk upload of content.

12. Conclusion

The AI-Powered Visual Book Generator meets the PRD's goals: it produces context-aware, visually reasonable book layouts significantly faster than manual typesetting, with the user in control at every page. Its defining decision — the LLM decides layout as validated JSON while Angular renders — makes the system secure,

deterministic, testable and inexpensive to evolve. The MVP is deliberately scoped, with every trade-off documented and a clear path to production.